GENERATIVE AI & AGENTIC AI · LESSON 3 OF 12

Calling LLM APIs from Python: Messages, Streaming, Structured Output, Costs and Reliability

Move from the chat window to code. Set up API keys safely, send messages with system prompts, hold multi-turn conversations, stream responses, get schema-valid JSON, control reasoning effort, cache long prompts, count tokens and estimate cost, batch large jobs, handle errors and retries, and run the same pattern with other providers, with a runnable offline simulator for practice.

GenAI Launchpad

✦ Anthropic Python SDK 1.x · Claude Opus 5.5 / Sonnet 5.5 / Haiku 5.5 (Oct 2026)

Intermediate

⏱ 120 min

By the end of this lesson you can

- Store API keys safely and create a client
- Send single and multi-turn requests with system prompts and read the response blocks
- Stream output and request structured, validated JSON
- Estimate and reduce cost with token counting, model choice, effort, caching and batching
- Handle rate limits, errors and retries and log usage

Datasets used

`customer_reviews.csv`

Setup

Text

```
pip install anthropic            # official SDK (also: openai, google-genai)
# Never paste keys into notebooks. Set an environment variable instead:
export ANTHROPIC_API_KEY="sk-ant-..."          # macOS/Linux
setx ANTHROPIC_API_KEY "sk-ant-..."            # Windows (new terminal afterwards)
# or put it in a .env file listed in .gitignore and load it with python-dotenv
```

Get a key from the provider console (console.anthropic.com, platform.openai.com, aistudio.google.com), set a monthly **spend limit**, and create separate keys for development and production.

Your first request

Python

```python
import anthropic
client = anthropic.Anthropic()                # reads ANTHROPIC_API_KEY from the environment

response = client.messages.create(
    model="claude-opus-5-5",                  # current default model; claude-sonnet-5-5 / claude-haiku-5-5 are cheaper
    max_tokens=16000,                         # upper limit on output tokens
    system="You are a concise data analyst. Answer in at most 3 bullet points.",
    messages=[{"role": "user", "content": "Why might a churn model have high accuracy but low recall?"}],
)
for block in response.content:                # the reply is a list of content blocks
    if block.type == "text":
        print(block.text)
print(response.stop_reason, response.usage.input_tokens, response.usage.output_tokens)
```

Field	Meaning
`model`	Which model answers
`system`	Standing instructions: role, rules, format
`messages`	The conversation: alternating `user` / `assistant` turns
`max_tokens`	Maximum length of the reply
`response.content`	List of blocks (`text` , `thinking` , `tool_use` ...); check `.type`
`response.stop_reason`	`end_turn` , `max_tokens` (cut off), `tool_use` , `refusal`
`response.usage`	Input and output tokens, which you pay for

> **NOTE** **Sampling parameters on new models**
>
> Older tutorials set `temperature`. Current Claude reasoning models manage sampling themselves (sending `temperature` to them returns an error); control depth and cost with `output_config={"effort": "low" | "medium" | "high" | "xhigh" | "max"}` instead. Check each provider's documentation for the model you use.

Practise offline: a simulated client

Real calls need a key and cost money. To learn the **shapes** of requests and responses, this lesson uses a small simulator with the same interface. Swap `FakeClient()` for `anthropic.Anthropic()` when you have a key.

Python

```python
import json, re, time, random, types, pandas as pd, numpy as np
class FakeClient:
    """Mimics client.messages.create(...) for offline practice: returns canned but
realistic responses and token usage."""
    def __init__(self, fail_first=0): self.calls, self.fail_first = 0, fail_first;
self.messages = self
    def create(self, model, max_tokens, messages, system=None, **kw):
        self.calls += 1
        if self.calls ⩽ self.fail_first: raise RuntimeError("429 rate_limit_error: please
retry after 1s")
        last = messages[-1]["content"] if isinstance(messages[-1]["content"], str) else
json.dumps(messages[-1]["content"])
        t = last.lower()
        if "json" in t or "output_config" in kw:
            neg = any(w in t for w in ["drain", "poor", "slow", "late", "worst", "regret",
"cheap"])
            text = json.dumps({"overall": "negative" if neg else "positive", "aspects": [a
for a in ["battery", "screen", "sound", "delivery", "price"] if a in t], "summary":
last[:60]})
        elif "my name is" in " ".join(m["content"] for m in messages if
isinstance(m["content"], str)).lower() and "name" in t:
            text = "Your name is " + re.search(r"my name is (\w+)", " ".join(m["content"]
for m in messages if isinstance(m["content"], str)), re.I).group(1) + "."
        else:
            text = "• Accuracy counts both classes; with few churners, predicting 'stay'
scores well.\n• Recall only counts churners caught.\n• Check precision/recall at the
decision threshold."
        n_in = int(len((system or "").split() + " ".join(str(m["content"]) for m in
messages).split()) * 1.3); n_out = int(len(text.split()) * 1.3)
        return types.SimpleNamespace(content=[types.SimpleNamespace(type="text",
text=text)], stop_reason="end_turn", model=model,
                                     usage=types.SimpleNamespace(input_tokens=n_in,
output_tokens=n_out, cache_read_input_tokens=0))
client = FakeClient()
r = client.messages.create(model="claude-opus-5-5", max_tokens=16000, system="You are a
concise data analyst.",
                           messages=[{"role": "user", "content": "Why might a churn model
have high accuracy but low recall?"}])
print(next(b.text for b in r.content if b.type == "text"))
print(r.stop_reason, r.usage.input_tokens, "in /", r.usage.output_tokens, "out tokens")
```

OUTPUT

```
• Accuracy counts both classes; with few churners, predicting 'stay' scores well.
• Recall only counts churners caught.
• Check precision/recall at the decision threshold.
end_turn 22 in / 32 out tokens
```

Multi-turn conversations

The API is **stateless**: send the whole history every time.

Python

```
history = []
def chat(user_text):
    history.append({"role": "user", "content": user_text})
    resp = client.messages.create(model="claude-opus-5-5", max_tokens=2000,
messages=history)
    reply = next(b.text for b in resp.content if b.type == "text")
    history.append({"role": "assistant", "content": reply})
    return reply
chat("My name is Asha and I analyse telecom churn.")
print(chat("What is my name?"))
print(len(history), "messages are sent with the next request")
```

OUTPUT

```
Your name is Asha.
4 messages are sent with the next request
```

Long conversations grow the input (and the bill) each turn: summarise or trim old turns, or use server-side compaction for very long sessions.

Streaming

Stream tokens as they are generated so users see progress (and long requests avoid timeouts):

Python

```
with client.messages.stream(
    model="claude-opus-5-5",
    max_tokens=64000,
    messages=[{"role": "user", "content": "Write a 300-word data-quality checklist for a
new dataset."}],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
    final = stream.get_final_message()          # full message with usage, when you need it
```

Structured output

Ask for JSON that matches a schema; the SDK validates it for you.

Python

```python
from pydantic import BaseModel
from typing import Literal, List

class ReviewAnalysis(BaseModel):
    overall: Literal["positive", "negative", "neutral"]
    aspects: List[str]
    summary: str

response = client.messages.parse(
    model="claude-opus-5-5",
    max_tokens=16000,
    messages=[{"role": "user", "content": "Analyse this review: 'Display is bright but the
battery drains fast.'"}],
    output_format=ReviewAnalysis,
)
result = response.parsed_output              # a validated ReviewAnalysis object
print(result.overall, result.aspects)
```

Python

```python
from pydantic import BaseModel, ValidationError
from typing import Literal, List
class ReviewAnalysis(BaseModel):
    overall: Literal["positive", "negative", "neutral"]
    aspects: List[str]
    summary: str
reviews = pd.read_csv("customer_reviews.csv").sample(5, random_state=2)
rows = []
for t in reviews.review_text:
    resp = client.messages.create(model="claude-haiku-5-5", max_tokens=1000, messages=
[{"role": "user", "content": f"Return JSON for this review: {t}"}])
    parsed = ReviewAnalysis.model_validate_json(resp.content[0].text)
    rows.append({"review": t[:55], "overall": parsed.overall, "aspects": parsed.aspects})
print(pd.DataFrame(rows).to_string(index=False))
```

OUTPUT

```
                                                review  overall         aspects
Bahut badiya. The app is really smooth though the touch positive              []
Happy with the purchase. Price is very reasonable for t positive         [price]
Loved it! scren is crisp even outdoors, the finish is c positive              []
Superb product. Screen is crisp even outdoors, price is positive [screen, price]
Setup took two minutes, build quality feels premium, sc positive        [screen]
```

Reasoning effort

Python

```
response = client.messages.create(
    model="claude-opus-5-5",
    max_tokens=16000,
    thinking={"type": "adaptive", "display": "summarized"},   # the model decides how much
to think
    output_config={"effort": "high"},                          # low | medium (default on
Opus 5.5) | high | xhigh | max
    messages=[{"role": "user", "content": "Our churn fell from 27% to 22% after a price cut
that also changed the customer mix. Can we credit the price cut?"}],
)
for b in response.content:
    print(b.type, (b.thinking if b.type == "thinking" else b.text)[:300])
```

Use **low** effort for simple, high-volume tasks (classification, extraction) and **high** for analysis, planning and code.

Cost: measure, estimate, reduce

Python

```
PRICE = {"claude-opus-5-5": (4.00, 20.00), "claude-sonnet-5-5": (2.00, 10.00), "claude-
haiku-5-5": (0.10, 0.50)}   # USD per million tokens (input, output), Oct 2026 list prices
def cost_usd(model, tokens_in, tokens_out): pin, pout = PRICE[model]; return (tokens_in *
pin + tokens_out * pout) / 1e6
job = {"reviews": 100_000, "tokens_in": 350, "tokens_out": 80}            # classify 1 lakh
reviews
usd_inr = 88
for m in PRICE:
    c = cost_usd(m, job["reviews"] * job["tokens_in"], job["reviews"] * job["tokens_out"])
    print(f"{m:18s} ${c:8,.2f}  ≈ ₹{c * usd_inr:10,.0f}  | with Batch API (50% off) ≈ ₹{c *
usd_inr / 2:9,.0f}")
```

OUTPUT

```
claude-opus-5-5    $  300.00  ≈ ₹    26,400  | with Batch API (50% off) ≈ ₹   13,200
claude-sonnet-5-5  $  150.00  ≈ ₹    13,200  | with Batch API (50% off) ≈ ₹    6,600
claude-haiku-5-5   $    7.50  ≈ ₹       660  | with Batch API (50% off) ≈ ₹      330
```

Prices change: check the provider's pricing page before budgeting. Main levers:

Lever	Saving
Right-size the model (Haiku/Sonnet for simple, high-volume steps)	Often 10–40×
Lower effort for simple tasks	Fewer thinking tokens
Prompt caching for long, repeated prefixes (system prompt, documents)	Cached input reads cost about 10% of normal input
Batch API for non-urgent jobs	50% off, results within 24 hours
Shorter prompts and outputs (ask for JSON, not essays)	Linear
Count tokens before sending (`client.messages.count_tokens(...)`)	Avoid surprises

Python

```python
# Prompt caching: mark the long, stable part of the prompt
response = client.messages.create(
    model="claude-opus-5-5", max_tokens=16000,
    system=[{"type": "text", "text": LONG_POLICY_DOCUMENT, "cache_control": {"type": "ephemeral"}}],
    messages=[{"role": "user", "content": "Does the refund policy cover damaged boxes?"}],
)
print(response.usage.cache_read_input_tokens)     # > 0 on repeat calls when the cache hits

# Token counting
print(client.messages.count_tokens(model="claude-opus-5-5", messages=[{"role": "user", "content": "Hello"}]).input_tokens)
```

Reliability: errors and retries

The SDK already retries connection errors, 429 (rate limit) and 5xx errors twice with backoff. Handle the rest explicitly:

Python

```python
import anthropic
try:
    response = client.messages.create(model="claude-opus-5-5", max_tokens=16000,
messages=msgs)
except anthropic.RateLimitError as e:          # 429: slow down, queue, or retry later
    retry_after = int(e.response.headers.get("retry-after", "30"))
except anthropic.BadRequestError as e:         # 400: fix the request (do not retry)
    print(e.message)
except anthropic.APIStatusError as e:          # other HTTP errors
    print(e.status_code, e.message)
except anthropic.APIConnectionError:           # network problem
    ...
if response.stop_reason == "max_tokens": ...   # output was cut off: raise max_tokens or
stream
if response.stop_reason == "refusal": ...      # declined for safety: check
response.stop_details
```

Python

```python
def call_with_retry(fn, tries=4, base=0.05):
    for attempt in range(1, tries + 1):
        try: return fn()
        except RuntimeError as e:
            if "429" not in str(e) or attempt == tries: raise
            wait = base * 2 ** (attempt - 1) + random.uniform(0, base); print(f"  attempt
{attempt} failed ({e}); retrying in {wait:.2f}s"); time.sleep(wait)
flaky = FakeClient(fail_first=2)
r = call_with_retry(lambda: flaky.messages.create(model="claude-haiku-5-5", max_tokens=500,
messages=[{"role": "user", "content": "hello"}]))
print("succeeded after", flaky.calls, "calls")
```

OUTPUT

```
  attempt 1 failed (429 rate_limit_error: please retry after 1s); retrying in 0.05s
  attempt 2 failed (429 rate_limit_error: please retry after 1s); retrying in 0.15s
succeeded after 3 calls
```

TIP **Production logging**

Log for every call: timestamp, model, prompt version, input/output tokens, latency, stop reason and the request id (`response._request_id`). This makes cost dashboards, debugging and audits possible. Never log secrets, and mask personal data.

Large jobs: the Batch API

Python

```python
from anthropic.types.message_create_params import MessageCreateParamsNonStreaming
from anthropic.types.messages.batch_create_params import Request
batch = client.messages.batches.create(requests=[
    Request(custom_id=f"review-{i}", params=MessageCreateParamsNonStreaming(
        model="claude-haiku-5-5", max_tokens=500, messages=[{"role": "user", "content":
f"Classify: {text}"}]))
    for i, text in enumerate(review_texts)])
# later: poll client.messages.batches.retrieve(batch.id).processing_status until "ended",
# then iterate client.messages.batches.results(batch.id) and match results by custom_id
(order is not guaranteed)
```

Other providers

The pattern is the same everywhere: a client, a model name, messages, and a response with text and usage. OpenAI (`openai` package, Responses/Chat Completions APIs), Google (`google-genai` , Gemini), and open models served locally with **Ollama** or in the cloud with providers such as Together or Groq. Libraries like LiteLLM offer one interface over many providers; keep the provider-specific features (structured outputs, caching, tools) in mind when switching.

Summary

KEY TAKEAWAYS

- Keep API keys in environment variables or a secrets manager; set spend limits.
- `client.messages.create(model, max_tokens, system, messages)` ; read `response.content` blocks, `stop_reason` and `usage` .
- The API is stateless: send the conversation history each call and manage its length.
- Stream long outputs; use `messages.parse` / structured outputs for validated JSON; use effort (and adaptive thinking) for hard tasks.
- Control cost with model choice, effort, prompt caching, batching, short outputs and token counting.
- Handle rate limits and errors, check stop reasons, and log usage per call.

Quick quiz

6 questions · tap an answer to check it

1. Where should an API key live?

- **A** In an environment variable or secrets manager, never in code
- **B** In the notebook
- **C** In the prompt
- **D** In a public repository

Answer: A. Rotate keys if exposed.

2. Why must you send the full message history each time?

- **A** The API is stateless; it remembers nothing between calls
- **B** To save tokens
- **C** It is optional
- **D** The API stores history automatically

Answer: A. Manage history length for cost.

3. `stop_reason == "max_tokens"` **means...**

- **A** The reply was cut off at the output limit
- **B** The model finished normally
- **C** A tool was called
- **D** The request failed

Answer: A. Increase max_tokens or stream.

4. Which is the best lever for classifying 1 lakh reviews overnight cheaply?

- **A** A small model with the Batch API
- **B** The largest model in real time
- **C** Higher temperature
- **D** Longer prompts

Answer: A. Batches are 50% cheaper.

5. What does prompt caching help with?

- **A** Repeated long prefixes such as a policy document or system prompt
- **B** Changing questions only
- **C** Image generation
- **D** Training the model

Answer: A. Cached reads are far cheaper.

6. Which errors should you generally retry?

- **A** Rate limits (429), server errors (5xx) and network errors
- **B** 400 bad requests
- **C** Authentication errors
- **D** Validation errors in your own code

Answer: A. Fix 4xx requests instead.

EXERCISE **Practice**

(1) Using the simulator (or a real key), classify 20 reviews and store results with token counts in a DataFrame. (2) Write a function that estimates monthly cost from requests per day and average tokens for each model. (3) Add logging of latency and tokens to `chat()`. (4) With a real key, compare the output of `claude-haiku-5-5` and `claude-opus-5-5` on five tricky reviews.

Show solution (worked answer)

Python

```
recs = []
for t in pd.read_csv("customer_reviews.csv").review_text.head(20):
    t0 = time.perf_counter(); r = client.messages.create(model="claude-haiku-5-5",
max_tokens=500, messages=[{"role": "user", "content": f"JSON: {t}"}])
    recs.append({"text": t, **json.loads(r.content[0].text), "in":
r.usage.input_tokens, "out": r.usage.output_tokens, "ms": round(1000 *
(time.perf_counter() - t0), 1)})
log = pd.DataFrame(recs)
def monthly_cost(req_per_day, tin, tout): return {m: round(cost_usd(m, 30 *
req_per_day * tin, 30 * req_per_day * tout), 2) for m in PRICE}
print(monthly_cost(5000, 800, 200))
```

INTERVIEW CORNER **Questions you may be asked**

Q: How would you estimate the cost of an LLM feature? Requests per day × (input tokens × input price + output tokens × output price), measured on real samples with token counting, then reduced with model choice, caching, batching and shorter outputs.

Q: How do you make LLM API calls robust? Timeouts, retries with exponential backoff for 429/5xx, validation of outputs, checking stop reasons, fallbacks (another model or a rule), idempotent processing and logging.

Q: How do you keep a long chatbot conversation within budget? Trim or summarise older turns, cache the stable system prompt, retrieve only relevant documents, and cap output length.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.